

Lab Practicum 01: Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

STUDENT NAME	STUDENT ID	HOST CPU / SOC MODEL	OPERATING SYSTEM & KERNEL
YERMURAT AINAT	230103161	INTEL	WINDOWS 11

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

TASK 1: Host Silicon Audit & Flynn’s Taxonomy Mapping15 Minutes | 20 Points

Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: lscpu && lscpu --caches macOS: sysctl -a grep machdep.cpu Windows: Run wmic cpu get /format:list and inspect Task Manager > Performance > CPU.			
Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Intel Core i5-11300H, Tiger Lake-H, x86-64	Base & Max Boost Clock	3.10 GHz / 4.40 GHz
Physical Hardware Cores	4 cores	Logical Cores (SMT/Threads)	8 threads
L1 Data / Instruction Cache	L1d: 192 KB L1i: 128 KB	L2 Cache (Per Core / Total)	5 MB total
L3 Shared Last-Level Cache	8 MB	SIMD Vector Extensions	AVX2, AVX-512

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:
My Intel Core i5-11300H system can operate in different Flynn's Taxonomy modes. SISD is used for single-threaded tasks, where one instruction works on one data stream. SIMD is used for vector operations, where one instruction processes multiple data elements at the same time using CPU vector extensions such as AVX2. MIMD is used when multiple CPU cores execute different instructions on different data at the same time, for example when the operating system runs multiple threads across the 4 physical cores and 8 logical threads.

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall35 Minutes | 30 Points

Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: Matrix Multiplication				Language & Runtime: C++ / OpenMP		
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	Efficiency EN = SN/N
N = 1 (Baseline)	0.965342	0.950939	0.94703	0.954451	1.00×	100.0%
N = 2	0.498982	0.517381	0.514733	0.510366	1.87×	93.5%
N = 4	0.297740	0.292006	0.302356	0.297367	3.21×	80.2%
N = 8	0.229424	0.227936	0.227086	0.228149	4.18×	52.3%
N = 16	0.230262	0.231235	0.250330	0.237275	4.02×	25.1%
N = 32	0.233555	0.233580	0.243671	0.236935	4.03×	12.6%

Q2.1: Scaling Saturation & Hardware Contention Analysis:
Scaling saturation occurs at approximately 8 threads. The speedup improves from 1.87× at 2 threads to 4.18× at 8 threads, but decreases slightly to 4.02× at 16 threads and remains almost unchanged at 32 threads. My CPU has 4 physical cores and 8 logical threads, so the best performance is reached around the hardware thread capacity. Beyond 8 threads, additional software threads introduce scheduling overhead and increased contention for CPU resources and cache, so they do not provide additional speedup.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap25 Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: $10 \times 1,000,000 = 10,000,000$. Execute 10 consecutive runs.

Run	Measured Output	Error	Run	Measured Output	Error
#1	1,872,377	8,127,623	#6	1,257,262	8,742,738
#2	1,327,960	8,672,040	#7	1,659,424	8,340,576
#3	1,519,267	8,480,733	#8	1,429,643	8,570,357
#4	1,612,691	8,387,309	#9	1,662,934	8,337,066
#5	1,435,403	8,564,597	#10	1,334,477	8,665,523

Q3.1: Read-Modify-Write (RMW) Window

The race condition occurs because the shared counter increment is a read-modify-write operation. At the machine-instruction level, one thread loads the current value from memory, adds 1, and then stores the new value back. If two threads read the same value before either stores its updated value, both perform the addition based on the same old value. One update overwrites the other, causing lost increments. This process repeats across multiple threads, so the final value is much lower than the theoretical 10,000,000.

Q3.2: The Synchronization Penalty

The unlocked version took 19.3902 ms, while the locked version took 404.7 ms. The synchronized version is substantially slower because every increment must acquire and release a mutex. Threads may have to wait when another thread owns the lock, creating serialization and synchronization overhead. Although the lock prevents data corruption and produces a correct result, it significantly reduces parallelism.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p)

T1 = 0.954451 s | T2 = 0.510366 s

Calculated p = 0.93056

2. Amdahl Theoretical Speedup Ceiling

Sequential fraction (1 - p) = 0.06944

Max Theoretical Speedup (N → ∞) = 14.40x

Q4.1: Strong vs. Weak Scaling Analysis:
Amdahl's Law assumes a fixed problem size, so the sequential fraction becomes the limiting factor as more cores are added. Even with 64 cores, the sequential portion cannot be parallelized, which limits the maximum speedup. Gustafson's Law assumes that the problem size grows with the number of processors. Therefore, more of the total workload can be executed in parallel, allowing the scaled speedup to approach linear growth. In this case, Amdahl predicts a much lower speedup because of the 6.95% sequential fraction, while Gustafson predicts approximately 59.62x scaled speedup for 64 cores.